

Skill 3 · Base de Conversão LSP → Java

Versão: v1.10 · Interna · skill-03-base-conversao-lsp-java.md

Skill interna — **não** é fluxo de usuário. Aplique as regras globais do Router. Em conflito de assinatura, **revalide na Skill 2 / página oficial**.

Fronteira: Skill 2 = links/alias; **este núcleo** = mecânica + tipagem + esqueletos + armadilhas; catálogo/acesso/exemplos = **referências** skill-03-referencia-* .

Precedência: docs oficiais Skill 2 → catálogo (referencia-catalogo) → padrões padrao_compilacao (Eclipse) → anexos sanitizados → validacao_manual .

Crescimento do catálogo confirmada : só com doc oficial / Skill 2. Achados de compilação entram como padrao_compilacao , não como inventados.

Como consultar (ordem obrigatória)

- A. Restrições + Regra de ouro + Invariantes ← nunca pular (neste arquivo)
- B. Workflow (6 passos)
- C. Tipos / sintaxe / VaPara / Funcao (se necessário)
- D. Catálogo – skill-03-referencia-catalogo (só a família)
- E. Acesso a dados – skill-03-referencia-acesso-dados (se cursor/SQL/USU_*)
- F. Armadilhas Eclipse + checklist (neste arquivo)
- G. Exemplos/goldens – skill-03-referencia-exemplos-goldens (só o análogo)
- H. Saída tipada para a Skill 1

Não leia o catálogo inteiro de ponta a ponta. Localize a **família** no índice. Em conflito de assinatura, **revalide na Skill 2**.

Índice rápido do arquivo

Precisa de...	Vá para
Proibições / minutos / Sem Situacao.getMinutos	Restrições absolutas
Modelo mental LSP→contexto	Regra de ouro
Passo a passo	Workflow
Tipagem / Se→if / Logico / Escolha / VaPara	Tipos e sintaxe
Esqueleto Apuracao / Consistência / BH	Esqueletos + tipos de regra
DatPro, HorSit, FPxMar, históricos...	referencia-catalogo → família
TipCon / MarcacaoRegra / overloads HorSit	referencia-catalogo + notas padrao_compilacao
Cursor USU_* / .sc / ContextSession	referencia-acesso-dados
Erros Eclipse / anti-padrões	Armadilhas
CalculaQtdMinutos / interjornada / TipCon SQL	referencia-exemplos-goldens
Templates IEntity / .sc / DBCenter	referencia-acesso-dados

VaPara / operadores	Tipos e sintaxe
Goldens G-SIT / G-CUR / G-USU / G-MIX	referencia-exemplos-goldens
InicioCalculo / AposGravar	Outros pontos
🚫 SDK / ⚠️ pendente	Itens pendentes (SDK)
Campos LSP conflitantes	Tabela conflitantes

Âncoras 80% (não leia o arquivo inteiro)

Se a regra tem...	Vá direto para
HorSit / situações / minutos	Catálogo → Situações + G-SIT
Marcações / FPxMar / MarcacaoAnterior	Catálogo → Marcações + Exemplos
TipCon / R034FUN	Campos conflitantes + TipCon via ContextSession
Cursor USU_* / interface / .sc	Acesso a dados + G-USU
Cursor R* / CriarCursor	Acesso a dados + G-CUR (API primeiro)
VaPara / labels	Tipos e sintaxe → VaPara
INSERT/UPDATE/DELETE	DBCcenter (não ContextSession)
Situação + R* + USU_* juntos	exemplos-goldens → G-MIX

Índice símbolo LSP → seção (busca rápida)

Símbolo / gatilho LSP	Seção
HorSit / setHorSit / situação / minutos	referencia-catalogo · G-SIT
FPxMar / FLeMar / MarcacaoAnterior / marcações	Catálogo → Marcações · Exemplos · CHK-MARANT
TipCon / R034FUN	catalogo (Campos) · exemplos TipCon
CriarCursor / AbrirCursor / R030EMP / R014SIN / R*	Acesso a dados · G-CUR (sem .sc)
USU_* / tabela custom	Acesso a dados · templates IEntity/.sc · G-USU
HorSit + R* + USU_* na mesma regra	G-MIX
DatPro / HorSis / data sistema	Catálogo → Data / sistema
TotSit / totalizador	Catálogo → Situações / totalizadores
VaPara / labels	Tipos e sintaxe → VaPara
Escolha / Caso / Logico	Tipos e sintaxe
End / múltiplos retornos	Exemplos End → retorno

ExecSQL / INSERT/UPDATE/DELETE	DBCenter · nunca Senior SQL 2
Mensagem(	Restrições · mensagemLog / exceção de domínio
InicioCalculo / AposGravar	Outros pontos de customização
TipoHoraExtra / @Transactional	Itens pendentes (SDK) 🚫
Método ausente no catálogo	Skill 2 Índice das Funções → validacao_manual + TODO (Skill 1)

Progressive disclosure (Agent Skills)

Este arquivo é o **núcleo** (<500 linhas). Detalhes sob demanda:

Precisa de...	Carregar
Catálogo de equivalência / famílias / TipCon / constantes	skill-03-referencia-catalogo.md
ICursor / .sc / ContextSession / DBCenter / IEntity	skill-03-referencia-acesso-dados.md
Exemplos sanitizados + goldens G-SIT/G-CUR/G-USU/G-MIX	skill-03-referencia-exemplos-goldens.md

Não leia as três referências de ponta a ponta — só a âncora necessária (índice símbolo no núcleo).

Fontes oficiais

Fonte	URL	Uso
Equivalência das funções de regras	https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/informacoes-adicionais/rotinas/gpo/integracao-controle-ponto-refeitorio/equivalencia-funcoes-regras.htm	Mapa oficial
Índice das Funções HCM 6.10.4	https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/customizacoes/funcoes.htm	Assinatura + contexto do método

Catálogo abaixo: evidência confirmada (salvo nota). Exemplos: padrao_anexo .

Assinatura/contexto incompletos no catálogo → abrir Índice das Funções (Skill 2) antes de gerar código.

Quando usar / não usar

Usar	Não usar
Converter/mapear variáveis e funções de apuração LSP→Java	Só mentoria; inventar método; só citar link (Skill 2)

Restrições absolutas

1. Preferir o catálogo; se faltar → Skill 2 → validacao_manual (não inventar).
2. Horas em APIs de ponto = **minutos inteiros** (int , não double solto).

3. SQL/cursor → API semântica do catálogo **antes** de EntitySession/ContextSession — exceto tabela **USU_* custom** sem API (aí ICursor+ .sc).
4. **Proibido** getSituacao(...).getMinutos() / setMinutos(...) — use getHorSit / setHorSit / zeraHorasSituacao .
5. Sanitize nomes de cliente em exemplos anexados; package do projeto é **parâmetro** (ex. sanitizado gestaopontoCustom) — nunca vaziar cliente.
6. Sem Mensagem() /popup em apuração — preferir mensagemLog .
7. Confirmar se o método existe no **contexto** da regra (apuração ≠ consistência ≠ BH ≠ geral).
8. execute() **sem** throws Exception ; auxiliares private no **nível da classe** (nunca aninhados).
9. ContextSession/DBCenter da plataforma ≠ Senior SQL 2 (SQL 2 continua proibido).

Regra de ouro

LSP e Java diferem no **modelo de execução**: variáveis/funções globais LSP → **métodos no objeto de contexto**.

Quem só troca Inicio/Fim por { } produz código que não compila.

Workflow (6 passos)

1. Identificar contexto (apuração / consistência / bloqueio / fechamento BH / geral).
2. Inventariar construções LSP.
3. Mapear no catálogo (nome **e** ordem de parâmetros).
4. Traduzir mecânica (getters/setters, arrays→métodos, cursor→ List , End →retorno).
5. Traduzir sintaxe (Se → if , = comparação→ == / .isEqual() , etc.).
6. Revisar armadilhas.

Invariantes

1. Toda variável/função de contexto LSP → chamada de método.
2. Não inventar assinatura.
3. Conferir ordem de parâmetros.
4. Horas = minutos inteiros.
5. Tipagem forte Java.
6. Sem popup em apuração.
7. Arrays indexados → métodos/coleções (não getApuDiu(1) inventado).
8. Método deve existir no **contexto** da regra.

Tipos e sintaxe

LSP	Java
Numero inteiro	int / long
Numero decimal	double
Alfa	String
Logico / lógico 0/1	boolean (0→false, 1→true)
Data	LocalDate (ou padrão do SDK — não misturar Joda + java.time sem necessidade)

Hora (minutos)	int minutos
marcação	MarcacaoRegra (padrao_compilacao; preferir a Marcacao genérica)
Se / Senao / Enquanto / Inicio/Fim	if / else / while / { }
Escolha / Caso / OutroCaso	switch (sem fall-through acidental) ou if/else encadeado
Vapara / labels	early return; break/continue; ou flag + do { } while(false)
Funcao sem end	private void no nível da classe
Funcao com 1 end	retorno tipado do método
Funcao com vários end	int[] / objeto (não closures aninhadas)
Numero[n] array	int[n+1] ou double[n+1] (LSP 1-based → Java 0-based)
Tabela	List<record> / classe interna
@ ... @	//
= comparação	== / .isEqual() / .equals()
<>	!=
decimal ,	.
e / ou / nao	&& / `
RestoDivisao(a,b)	a % b
Mensagem(...) em regra	exceção de domínio (BusinessException / RegraApuracaoException) — não popup

Escolha / Caso (padrao_compilacao)

```
Escolha (n0pcao)
  Caso 1
    @ ...
  Caso 2
    @ ...
  OutroCaso
    @ ...
FimEscolha;
```

```
switch (n0pcao) {
  case 1:
    // ...
    break;
  case 2:
    // ...
    break;
```

```
    default:
        // OutroCaso
        break;
}
```

Alfa/ String : preferir `if/else` com `.equals()` (ou `switch` em Java 17+ com cuidado). **Não** copiar fall-through do `switch` sem `break` a menos que o LSP caia propositalmente no próximo `Caso` .

VaPara — 3 padrões (`padrao_compilacao`)

1. Early return

```
Se (condicao) { VaPara FimRegra; }
@ lógica
FimRegra:
```

```
if (condicao) return;
// lógica
```

2. Flag + do-while false (bloco “pulado” com label)

```
boolean fezProrrogacao = false;
do {
    // bloco principal
    fezProrrogacao = true;
} while (false);
if (!fezProrrogacao) { /* bloco do label */ }
```

3. Break de loop

```
while (condicao) {
    if (encerrar) break;
}
```

Para (0-based em Java)

```
for (int i = 0; i < n; i++) { /* LSP Para costuma 1..n */ }
```

Instruções

1. Seguir “Como consultar” (A-H)
2. Identificar contexto
3. Buscar item só na família do Catálogo
4. Assinatura/contexto duvidosos → Índice das Funções (Skill 2)
5. Ausente → `validacao_manual`

Tipos de regra e esqueletos (padrao_compilacao)

Tipo LSP	Classe base	Contexto	
Apuração	custom.senior.apuracao.Apuracao	getContextoApuracao()	Regra
Consistência	custom.senior.apuracao.ConsistenciaAcertos	getContextoConsistenciaAcerto()	Regra
Manutenção BH	custom.senior.bancohoras.RegraManutencaoBH	getContextoManutencaoBH()	Regra
Fechamento BH	custom.senior.bancohoras.RegraFechamentoBH	getContextoFechamentoBH()	Regra

Package: o informado pelo usuário/projeto. Exemplo sanitizado: `gestaopontoCustom` (não `gestaoDoPontoCustom`).

Apuração

```
@Rule(description = "Regra de Apuracao")
public class RegraApuracao extends Apuracao {
    @Override
    public void execute() { // SEM throws Exception
        ContextoGeralRH ctxGeral = getContainer().getContextoGeral();
        ContextoApuracao ctx = getContainer().getContextoApuracao();
    }

    private void auxiliar(ContextoApuracao ctx, int param) { /* nível da classe */ }
}
```

Consistência de Acertos

```
@Rule(description = "Consistencia de Acertos")
public class RegraConsistencia extends ConsistenciaAcertos {
    @Override
    public void execute() {
        ContextoConsistenciaAcerto ctx =
getContainer().getContextoConsistenciaAcerto();
        // extras: getHorSitAnterior, getDataInicial/Final, getHorSitAnteriorFaixa
    }
}
```

Fechamento BH

```
@Rule(description = "Fechamento BH")
public class RegraFechamentoBH extends custom.senior.bancohoras.RegraFechamentoBH {
    @Override
    public void execute() {
        ContextoFechamentoBH ctx = getContainer().getContextoFechamentoBH();
    }
}
```

```
}
}
```

Imports frequentes (padrao_compilacao): `java.time.LocalDate / LocalDateTime` ,
`com.senior.rule.Rule` , `com.senior.dataset.ICursor` , `MappedParamProvider` , `Colaborador` ,
`MarcacaoRegra` , `TipoIntervalo` , `TipoHoraExtra` , `ContextSession` , `DBCenter` , `ResultSet` ,
`com.senior.dataset.IEntity` (+ `@Entity / @Field`), `readonly IR030EMP / IR060DSI /...`

Armadilhas práticas

Armadilha / erro Eclipse	Correção
Só trocar sintaxe sem mapear variáveis	Inventário + getters/setters primeiro
Copiar ordem de parâmetros LSP	Confirmar no Índice das Funções
<code>getSituacao().get/setMinutos</code>	<code>getHorSit / setHorSit / zeraHorasSituacao</code>
Cursor SQL por reflexo	Buscar família no catálogo; <code>USU_*</code> → <code>ICursor</code>
<code>getHorSit(variavelDeMinutos)</code>	1º arg = código da situação
Inventar <code>getApuDiu(1)</code>	<code>getHoras / getHorasSeparadas</code>
<code>execute()</code> throws <code>Exception</code>	Remover <code>throws</code> (<code>IRulePoint</code>)
Método aninhado / código solto	Auxiliares no nível da classe
<code>col.getTipCon()</code>	SQL <code>R034FUN.TIPCON</code>
<code>MarcacaoAnterior.getHora/getData</code>	<code>diferencaMinutos</code>
<code>LocalDateTime</code> → <code>LocalDate</code>	<code>.toLocalDate()</code> em <code>MarcacaoRegra.getData()</code>
<code>.sc</code> em tabela <code>R*</code>	Remover <code>.sc</code> ; <code>ContextSession</code>
<code>.sc</code> não-JSON / BOM	Reescrever começando com {
<code>import com.senior.g5...IEntity</code>	<code>com.senior.dataset.IEntity</code>
Package <code>gestaoDoPontoCustom</code>	Package do projeto (ex. <code>gestaopontoCustom</code>)
<code>ResultSet</code> base-1	base-0
Filename ≠ classe pública	Renomear arquivo (Windows: temp rename)
Inventar método	<code>validacao_manual</code>
Misturar <code>java.time</code> e <code>Joda</code>	Preferir <code>java.time</code> / SDK

Mensagens Eclipse literais (referência)

Mensagem	Correção
The import <code>com.senior.g5</code> cannot be resolved	<code>import</code> <code>com.senior.dataset.IEntity</code>

execute() throws Exception not declared by IRulePoint	Remover throws Exception
The method getTipCon() is undefined for the type Colaborador	SQL R034FUN.TIPCON
The method getHora() is undefined for the type MarcacaoAnterior	diferencaMinutos
Type mismatch: cannot convert LocalDateTime to LocalDate	.toLocalDate()
Expected BEGIN_OBJECT but was STRING	.sc JSON puro começando com {
Tabela Rxxx inválida. É permitido mapear somente tabelas de usuário	Sem .sc em R*
The public type X must be defined in its own file	Filename = nome da classe

Windows (case): Refactor → nome temporário → nome correto (ou `ren` em duas etapas no CMD).

Saída para a Skill 1

```
contexto: Apuracao | FechamentoBH | outro | indefinido
item_lsp: ...
equivalente_java: ...
evidencia: confirmada | padrao_compilacao | padrao_anexo | validacao_manual
fonte: equivalencia-funcoes-regras | indice-funcoes | compilacao | anexo
limite: ...
```

Relacionados

Skill 2 (URLs/aliases) · Skill 1 · Skill 5 · referências `skill-03-referencia-catalogo.md` · `skill-03-referencia-acesso-dados.md` · `skill-03-referencia-exemplos-goldens.md`